PRACTICAL - 08

Aim: Guarded calculator (DivideByZeroException.java + Driver.java)

Code Screenshot:

DivideByZeroException.java

```java
public class DivideByZeroException extends Exception {
    public DivideByZeroException(String message) {
        super(message);
    }
}
```

Driver.java

```java
import java.util.Scanner;

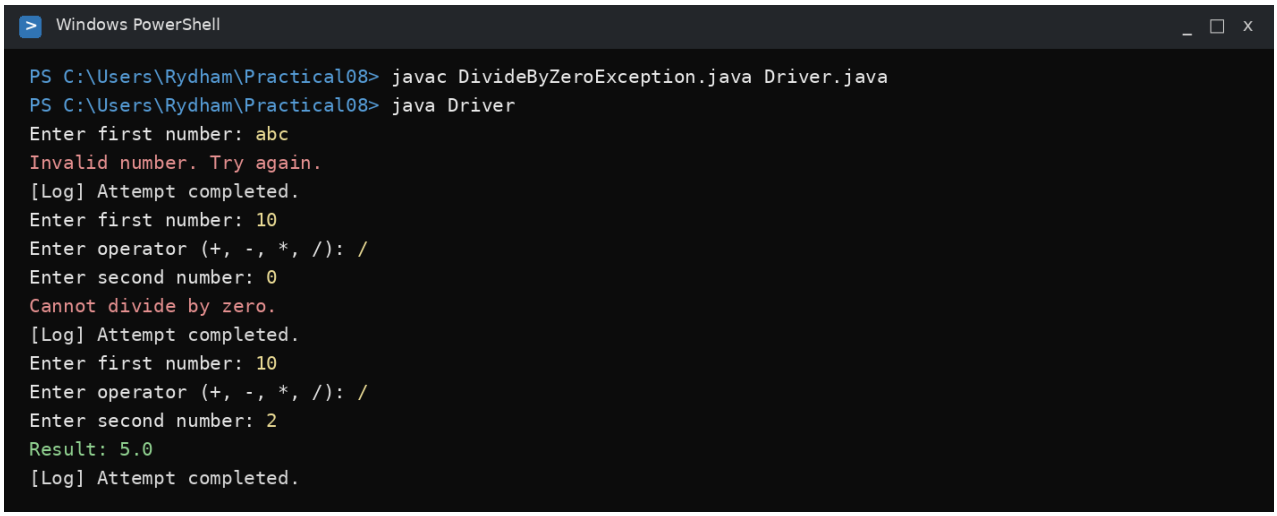
public class Driver {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean done = false;

        while (!done) {
            try {
                System.out.print("Enter first number: ");
                double a = Double.parseDouble(sc.nextLine());
                System.out.print("Enter operator (+, -, *, /): ");
                String op = sc.nextLine().trim();
                System.out.print("Enter second number: ");
                double b = Double.parseDouble(sc.nextLine());

                double result;
                switch (op) {
                    case "+" -> result = a + b;
                    case "-" -> result = a - b;
                    case "*" -> result = a * b;
                    case "/" -> {
                        if (b == 0) {
                            throw new DivideByZeroException("Cannot divide by zero.");
                        }
                        result = a / b;
                    }
                    default -> throw new IllegalArgumentException("Invalid operator.");
                }

                System.out.println("Result: " + result);
                done = true;
            } catch (NumberFormatException e) {
                System.out.println("Invalid number. Try again.");
            } catch (DivideByZeroException | IllegalArgumentException e) {
                System.out.println(e.getMessage());
            } finally {
                System.out.println("[Log] Attempt completed.");
            }
        }
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to create custom checked exceptions extending Exception.
- Mastered multi-catch blocks for differentiated error handling.
- Understood the execution guarantees of the finally block.
- Implemented resilient retry loops for user-driven input.
- Avoided abrupt program termination through structured exception handling.

Aim: Stock issue (Warehouse.java + Driver.java)

Code Screenshot:

Warehouse.java

```java
class OutOfStockException extends Exception {
    private int shortfall;

    public OutOfStockException(String item, int shortfall) {
        super(item + " is not available in the required quantity.");
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}

class InvalidQuantityException extends Exception {
    public InvalidQuantityException(String message) {
        super(message);
    }
}

public class Warehouse {
    private String[] items = {"Pen", "Book", "Bag"};
    private int[] stock = {5, 3, 2};

    public void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0) {
            throw new InvalidQuantityException("Quantity must be greater than zero.");
        }

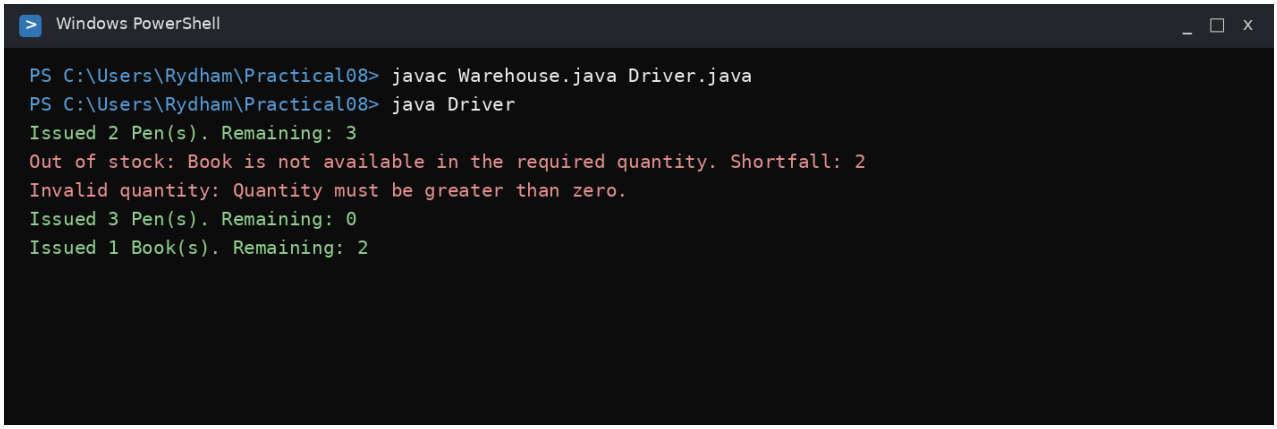
        for (int i = 0; i < items.length; i++) {
            if (items[i].equalsIgnoreCase(item)) {
                if (qty > stock[i]) {
                    throw new OutOfStockException(item, qty - stock[i]);
                }
                stock[i] -= qty;
                System.out.println("Issued " + qty + " " + item
                        + "(s). Remaining: " + stock[i]);
                return;
            }
        }
        System.out.println("Item not found: " + item);
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Warehouse warehouse = new Warehouse();
        String[] items = {"Pen", "Book", "Bag", "Pen", "Book"};
        int[] quantities = {2, 5, 0, 3, 1};

        for (int i = 0; i < items.length; i++) {
            try {
                warehouse.issue(items[i], quantities[i]);
            } catch (OutOfStockException e) {
                System.out.println("Out of stock: " + e.getMessage()
                        + " Shortfall: " + e.getShortfall());
            } catch (InvalidQuantityException e) {
                System.out.println("Invalid quantity: " + e.getMessage());
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to design exceptions that carry rich contextual error state.
- Understood checked vs unchecked exceptions and when to use each.
- Implemented fault-tolerant batch processing over item requests.
- Learned proper exception propagation and handling boundaries.
- Gained insights into enterprise inventory management error strategies.

Aim: AutoCloseable resource (WorkSession.java + Driver.java)

Code Screenshot:

WorkSession.java

```java
public class WorkSession implements AutoCloseable {
    public WorkSession() {
        System.out.println("Resource opened.");
    }

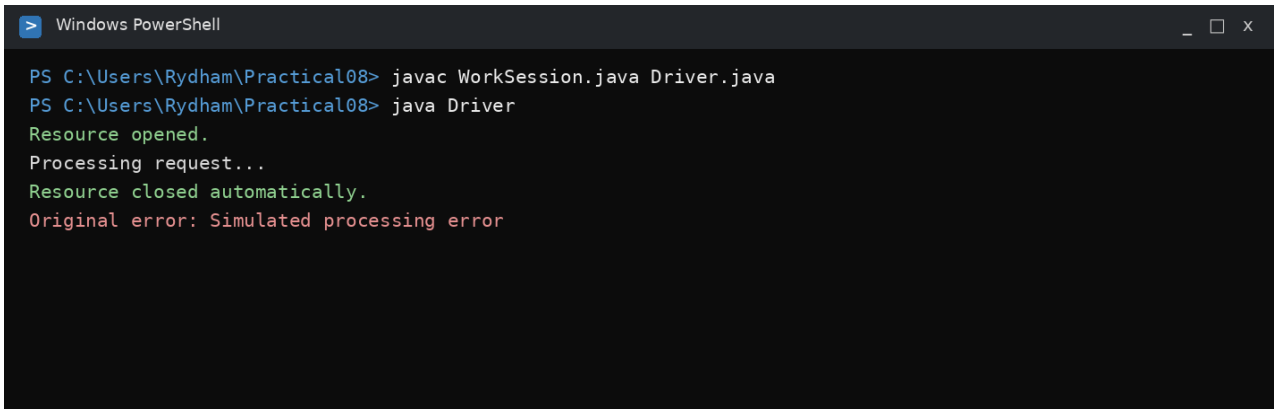
    public void process() {
        System.out.println("Processing request...");
        throw new IllegalStateException("Simulated processing error");
    }

    @Override
    public void close() {
        System.out.println("Resource closed automatically.");
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        try (WorkSession session = new WorkSession()) {
            session.process();
        } catch (IllegalStateException e) {
            System.out.println("Original error: " + e.getMessage());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to implement AutoCloseable resources in Java.
- Understood automatic resource closure using try-with-resources.
- Verified that close() executes when an exception occurs.
- Confirmed that the original exception remains reported.
- Practiced reliable resource management and exception handling.